TECHNICAL DOCUMENT

Capacity Planning Overview

Business Central Control Add-in — Architecture, Data Flow & DHTMLX Integration

Extension: DailyOptimizer (Optimizers) — app id range 50600–60700

Page: 50722 "Capacity Planning Overview" | Controladdin: DHXCapacityPlanningOverviewAddin

Codeunit: 50604 "DHX Data Handler" — region CPO_

Reference prototype: KLAAS/CPO_v131 (DHTMLXtempv112-app.js / DHTMLXtempv112-data.js)

Folder: src\dhx\capacity_planning_overview\

Document date: 2026-09-04 (supersedes the 2026-09-03 revision)

Revision note (2026-09-04)

This revision documents a same-day follow-up session on top of the 2026-09-03 cross-work-order-scoping/pagination work already described throughout this document: (1) reschedule (Section 2 drag, Section 3 click-to-relocate) was redesigned to run **entirely client-side, with zero BC round-trips, until the user clicks a new "Confirm changes" button** — see §3.1; (2) a processing-indicator (loading spinner) UI was added for every action that genuinely takes time — see §3.2; (3) Section 4's scrollbar-sync fix (§13.5) had a follow-on bug (§13.7) and a separate, unrelated performance bug (§13.8), both fixed; (4) an architectural finding about how InvokeExtensibilityMethod actually behaves (§13.9) that explains why several of these fixes work the way they do; (5) Section 4 now opens fully collapsed by default (§13.11) after two other performance approaches (DHTMLX Scheduler's smart_rendering, a full swap to the DHTMLX Gantt library) were tried and measured live, then set aside. Every changed section below is marked **(2026-09-04)** inline; everything else is unchanged from the prior revision.

Table of Contents

1. Overview & Purpose

The Capacity Planning Overview add-in is a Business Central control add-in that gives a single-screen view, for one selected Work Order, of: (a) whether that Work Order's own demand can realistically be covered by the company's resource pool day-by-day, (b) the Work Order's own Day Planning schedule, (c) a daily capacity-vs-request bar chart, and (d) a tree of every other Work Order's competing demand in the same visible window.

It is a faithful, BC-native port of a standalone DHTMLX prototype (`KLAAS\CPO_v131\DHTMLXtempv112-app.js`), with two deliberate substitutions: (1) the prototype's trial/evaluation `@dhx/trial-scheduler` npm package is replaced by this repository's already-licensed `dhtmlxscheduler.js`, and (2) all data is now real Business Central records instead of a static mock JSON file. The prototype's own client-side algorithms (shortage/coverage max-flow engine, tree builder, daily bar aggregator) are ported near-verbatim — Business Central's job is reduced to producing one JSON payload shaped exactly like the prototype's own mock `window.DHTMLXPlannerData` object.

> **Guiding principle (explicit, non-negotiable project requirement)**
> A BC control add-in binds exactly one JS component via one wrapper.js. All four visual sections, the shared header, and the "Days to show" control therefore live inside ONE JS class (`CapacityPlanningOverview`), not four separate `usercontrol` blocks. AL/Business Central is treated purely as a data source and event sink — "consider BC Add-ins as a browser for JS". The **2026-09-04 confirm-gated redesign (§3.1) sharpens this further**: AL is now not even an event sink for every interaction — it is a data source on load, and an event sink only for the one action the user deliberately commits to.

2. Architecture — BOOT / Wrapper / Controladdin Pattern

This follows the same skeleton every other DHTMLX add-in in this repository uses (e.g. `projectschedule`, `request_assignment`):

- BC loads the controladdin's `StartupScript`, which calls `BOOT()`.
- `BOOT()` (in wrapper.js) finds the host `<div id="controlAddIn">`, normalizes it to fill 100% width/height, renames its id to `cpo-root`, instantiates `new CapacityPlanningOverview('cpo-root')` (stored at `window.__cpo`), shows the full-page loading overlay (§3.2), and calls `Microsoft.Dynamics.NAV.InvokeExtensibilityMethod("ControlReady", [])`.
- AL's `ControlReady` trigger (page 50722) fires, which builds the initial JSON payload and calls `CurrPage.DhxCpo.SetPlanningData(json)`.
- `SetPlanningData` (the controladdin procedure) invokes the JS-side global function of the same name (in wrapper.js), which parses the JSON and calls `window.__cpo.applyPlanningData(json)`.
- `applyPlanningData` is the single entry point that fans out to every section's own `render*` method, then hides the loading overlay as its true last step (§3.2) — the user's actual "processing is done, ready for the next action" signal.

2.1 File Inventory

File	Role
DHXCapacityPlanningOverviewAddin.ControlAddin.al	Controladdin declaration: script/stylesheet load order, procedures (AL→JS), events (JS→AL), sizing properties. Unchanged in the 2026-09-04 session — the confirm-gated redesign changed WHEN the JS side calls the existing OnRescheduleWorkOrder event, not the event's own signature.

File	Role
page_50722_CapacityPlanningOverview.al	Card page hosting the single usercontrol. No SourceTable. Holds GWorkOrderNo / DaysToShow state, rebuilds and pushes the payload via RefreshData(). OnRescheduleWorkOrder's stub Message() dialog was removed (2026-09-04, §13.10) — it is now a true silent no-op stub, matching OnRequestCapacityLookup/OnSequenceChipClick.
startupScript.js	One line: BOOT();
wrapper.js	Thin BOOT/binding shell — instantiates the JS class, exposes the AL-facing globals (SetPlanningData / SetColors / LoadCapacityLookup / AppendOtherWorkOrderData), and (2026-09-04) owns the full-page loading overlay's safety timer (§3.2).
capacityPlanningOverview.js	The single JS component (~1,850 lines). All four sections, the shared header, the shortage/coverage engine, scroll-sync, tooltips, the processing-indicator methods, and (2026-09-04) the Confirm-button / unconfirmed-change state machine.
style.css	All layout/heat-map/chip/scrollbar CSS for the four sections and shared header, plus (2026-09-04) the spinner/overlay and Confirm-button CSS.
codeunit_50604_DHXDataHandler.al	Shared data-handler codeunit (also used by other DHX add-ins). Region CPO_ builds the one combined JSON payload.
Pag50662.WorkorderCard.al	Launching page — action CapacityPlanningOverviewAct calls SetWorkOrderNo() then Run() (top-level navigation, not a modal dialog).

Vendor libraries (dhtmlxscheduler.js/.css, suite.js/.css, GlobalFunction.js) are shared, root-level copies reused by every DHX add-in — no per-folder vendor duplication.

2.2 Controladdin Contract

```
controladdin DHXCapacityPlanningOverviewAddin
{
    RequestedHeight = 1150; MinimumHeight = 420;   VerticalShrink/Stretch = true;
    RequestedWidth = 1800; MinimumWidth = 900;     HorizontalShrink/Stretch = true;

    Scripts = dhtmlxscheduler.js, suite.js, GlobalFunction.js,
              capacityPlanningOverview.js,   // component class BEFORE wrapper.js
              wrapper.js;                    // BOOT() references the class - must load last

    StartupScript = startupScript.js;
    StyleSheets = dhtmlxscheduler.css, suite.css, style.css;

    procedure SetPlanningData(PlanningDataJsonTxt: Text);
    procedure SetColors(ColorsJsonTxt: Text);
    procedure LoadCapacityLookup(CapacityLookupJsonTxt: Text);
    procedure AppendOtherWorkOrderData(OtherWorkOrderDataJsonTxt: Text);
    procedure NotifyOtherWorkOrderDataTaskPending();
    procedure StopOtherWorkOrderDataPolling();

    event ControlReady();
    event OnRescheduleWorkOrder(DayShift: Integer; PayloadJsonTxt: Text);   // now fired ONLY
 by Confirm - see 3.1
    event OnRequestCapacityLookup(FilterJsonTxt: Text);
    event OnSequenceChipClick(PayloadJsonTxt: Text);
    event OnDaysToShowChanged(NumberOfDays: Integer);
    event OnPollOtherWorkOrderDataResult();
}
```

`RequestedHeight = 1150` is sized to the worst case: section 1 (~104px fixed) + section 2 (capped 400px) + section 3 (~180px) + section 4 (capped 420px) + ~40px chrome. `MinimumHeight = 420` with `VerticalShrink = true` lets a small Work Order shrink the control naturally instead of leaving dead space.

3. End-to-End Data Flow

```
Workorder Card (page 50662)
  action CapacityPlanningOverviewAct
    CPO.SetWorkOrderNo(Rec."Work Order No.")
    CPO.Run()                                    plain top-level nav, NOT RunModal()

page 50722 "Capacity Planning Overview"
  trigger ControlReady()
    EnsureDaysToShow()                           DaysToShow default = 30
    RefreshData()

  local procedure RefreshData()
    PlanningDataJson := DHXDataHandler.CPO_BuildPlanningDataJson_Paged(...)
    CurrPage.DhxCpo.SetPlanningData(PlanningDataJson)
                                                  (controladdin procedure -> JS global fn, sa
me name)

wrapper.js
  window.SetPlanningData(json) { window.__cpo.applyPlanningData(JSON.parse(json)); }

capacityPlanningOverview.js
  applyPlanningData(json)
    this.db = json; this.dates = [...]; this.woAnchor = 0; this._hasUnconfirmedChanges = fals
e
    this._baseRequests = this.aggregateRequests()
    this._baselineWithoutWO = this.aggregateOutstandingRequestsWithoutWO()
    renderWoSummaryScheduler(db)    Section 1
    renderWoScheduler(db)           Section 2 (ends by calling renderWorkOrder(), which also
                                             does a first-pass render of Section 3)
    measuredColumnWidth = measureColumnWidth()   needs a REAL rendered cell from Section 2
    renderCapacityBars(db)          Section 3 (re-rendered with the correct measured width)
    renderCentralTree(db)           Section 4
    bindScrollSync()                wires the shared horizontal scrollbar to all 4 sections
    updateConfirmButtonState()      resets the Confirm button to disabled - fresh server data
    hideLoading()                   true last step - see 3.2
```

Round-trip back to AL happens on three occasions (updated 2026-09-04 — was previously two immediate occasions plus a stub; is now genuinely gated on the two data-changing ones):

- "Days to show" changed (JS-owned input, top bar) → `OnDaysToShowChanged(NumberOfDays)` → AL sets DaysToShow and calls RefreshData() again (full round-trip, new payload) — unchanged; this genuinely needs new data the client doesn't have.
- **The Confirm button** (new, 2026-09-04) → `OnRescheduleWorkOrder(woAnchor, payloadJson)` — the ONE deliberate point any reschedule work (Section 2 drag, Section 3 click-to-relocate) reaches AL at all. Still a stub server-side (the real transactional Validate("Plan Date", ...) loop is a documented later step) — see §3.1.
- Section 4 chip click → `OnSequenceChipClick(payloadJson)` — stub, intended to open "Day Plannings" filtered to that specific line. Left as an immediate call (explicit 2026-09-04 decision, §3.1) — it is a navigate/view

action, not a schedule edit, so it is not part of the confirm-gated flow.

Section 3's Expand / Exp. to Task / Collapse buttons, the Capacity Lookup double-click, Section 2's own drag, and Section 3's click-to-relocate are all pure client-side, no AL round-trip per action — see §3.1 for the two that changed this session.

3.1 Confirm-Gated Reschedule (2026-09-04)

Prior behavior: every single Section 2 drag AND every Section 3 click-to-relocate immediately called `InvokeExtensibilityMethod('OnRescheduleWorkOrder', [shift, json])` — one real BC round-trip per intermediate mouse action, even though the trigger was (and still is) a complete no-op stub, and even though every value that call needed was already known client-side before it fired.

> **User-driven redesign**
> Stated directly by the user across a series of messages in the same session: "as long as no confirm data from JS then you must use in JS calculation/process… add confirm button in same row of title… if user clicked that button then you start round-trip with BC to update data"; confirmed again as "all action in JS must calculated in JS, do not act round-trip with BC" and "so no more round-trip with bc as long as no clicked on confirmed button". Final shape, confirmed against all four sections explicitly: Section 1 is a calculated board that always runs in JS; Section 2 is the Work Order to be modified (shiftable left/right), no round-trip until Confirm; Section 3 is user-driven and drives Section 2, also no round-trip until Confirm; Section 4 is static existing-data context (excluding this Work Order's own lines — §11) and is never touched by reschedule at all.

New client-side state machine (all in capacityPlanningOverview.js):

Method	Role
moveWorkOrderToDay(dayIndex)	Section 3 click-to-relocate. Updates woAnchor, resets anchor-dependent caches, calls renderWorkOrder() (Sections 1-3 re-render), then markUnconfirmedChange(). No AL call.
onEventChanged (Section 2 drag handler)	Same shape — computes the day-shift from the drop position, updates woAnchor if valid, re-renders, calls markUnconfirmedChange(). No AL call.
markUnconfirmedChange()	Sets this._hasUnconfirmedChanges = true and calls updateConfirmButtonState() — the only side effect of any local move.
updateConfirmButtonState()	Enables/disables #cpo-confirm-btn and toggles its .cpo-confirm-btn-pending CSS class (muted/disabled → solid blue "click me" look) to match the current flag.
bindConfirmButton()	Wires the button's click handler — calls confirmChanges() only if there is actually something unconfirmed.
confirmChanges()	The ONE deliberate BC round-trip point. showLoading(), InvokeExtensibilityMethod('OnRescheduleWorkOrder', [this.woAnchor, json]) with the CURRENT (possibly several-moves-later) woAnchor sent ONCE, then clears the pending flag, updates the button, hideLoading().

`#cpo-confirm-btn` ("Confirm changes") lives in the top bar, same row as the title, right of `#cpo-title` and left of the "Days to show" input — per the user's explicit placement request. It starts disabled/muted on every fresh load (`applyPlanningData` resets `_hasUnconfirmedChanges` to false and calls `updateConfirmButtonState()`), since a fresh server payload has nothing unconfirmed by definition.

Section 1 and Section 4, confirmed unaffected: Section 1 was already pure JS (renderWorkOrder() calls woSummaryScheduler.setCurrentView(...) to re-read the current woAnchor/caches — no AL involvement, before or after this redesign). Section 4 was already never re-rendered by any reschedule path (renderWorkOrder() does not call renderCentralTree()) — both facts predate this session and are unchanged by it; this redesign only removed the AL call that used to sit at the END of the Section 2/3 render paths.

3.2 Processing Indicators (2026-09-04)

New UI feedback for every action that genuinely takes time, added on explicit request ("the data load and processing take a time… show processing animated"). Both use the same visual language: a ring spinner (`.cpo-spinner`, a conic-gradient dark arc on a light track, CSS `@keyframes cpo-spin`), at two sizes/contexts.

Indicator	Shown for	Hidden by
#cpo-loading-overlay (full-host, blocking, .cpo-loading-overlay)	Initial ControlReady round-trip; a "Days to show" reload; Expand/Collapse-all's DHTMLX rebuild (via runBusy).	applyPlanningData's last step (hideLoading()); a 3-minute safety timer guards against it ever sticking.
#cpo-bg-loading ("Loading more data…", top bar, .cpo-inline-loading, non-blocking)	The background other-Work-Order-data pagination poll (§12) while it is still running.	StopOtherWorkOrderDataPolling (a result arrived) or the poll's own 30s ceiling.

`runBusy(fn)` exists specifically for GENUINELY slow (multi-second) synchronous CPU work with no natural browser yield point — today, only setTreeOpenState's Expand/Collapse-all rebuild qualifies. It shows the overlay, waits two `requestAnimationFrame` callbacks (forcing one real paint before the heavy work runs — showing the overlay and running the work in the same tick would never let the browser paint it at all), runs the work, then hides. It is deliberately NOT used around reschedule (moveWorkOrderToDay/the drag handler) any more — see §13.9 for why plain showLoading()/hideLoading() around a fire-and-forget InvokeExtensibilityMethod call never needed this trick, and why reschedule's own ~100-150ms local render cost is not worth a spinner flash for at all now that it has no BC call to wait on.

4. The JSON Payload Contract

Built entirely by codeunit 50604.CPO_BuildPlanningDataJson_Paged(WorkOrderNo, NumberOfDays, MaxOtherLines, out RemainingGroupKeys), shaped to mirror the reference prototype's own mock window.DHTMLXPlannerData object field-for-field. This is the only data BC ever pushes into the JS component on load — every section's render function derives everything else (shortage %, chip layout, bar heights, tree grouping) from this one object client-side, and (2026-09-04) every subsequent user interaction short of Confirm derives its results from this same object too, never re-fetching it.

Field	Type	Meaning / Source
daysToShow / startDate / endDate	int / ISO date	Echoed back so the JS-owned input can sync to whatever AL actually used. endDate = startDate + daysToShow - 1.
workdays[]	ISO date[]	Every calendar day in range (weekends included, not skipped) — the reference's own convention. Also doubles as the implicit workday-offset anchor for Section 2's bars.
workOrder.no / .description / .notEarlierThan / .notLaterThan	obj	The inspected Work Order's own header. description feeds the JS title bar directly.
project.no / .description	obj	The Work Order's parent Job ("Project No.").
skills[]	array	One entry per distinct Skill demanded (union of the inspected WO's own skills + every other WO's, §11). Each: code, color, textColor, border, dark, light.
resources[]	array	Every resource holding any demanded skill, capped at 15/skill for max-flow performance. Each: name (= Resource No.), skills[].

Field	Type	Meaning / Source
baseCapacity	number	Flat 8 (hours/resource/day).
externalFree[]	number[]	One entry per day in range — summed real "Res. Capacity Entry".Capacity for the capped resource pool with a non-blank Vendor No.
groups[]	array	Section 4's tree source. One entry per distinct Skill demanded by every OTHER Work Order in this window. Each has details[] = one entry per distinct Job No./Job Task No.
dayPlanningLines[]	array	Real Day Planning rows: the inspected WO's own lines plus every other WO's lines in the same window. Each carries a normalized workOrderNo.
workOrderSequences[]	array	Section 2's source. One entry per distinct Skill+Job+Task+SequenceNo belonging ONLY to the inspected Work Order, plus a per-(sequence, workday-offset) requested-hours map.

dayPlanningLines[] — per-line shape

```
{
  id, workOrderNo, job, task, description, sequenceLineNo,
  requestDate, requestedStartTime, requestedEndTime, requestedHours, requestedSkill,
  assignedResourceNo, assignedDate, assignedStartTime, assignedEndTime, assignedHours,
  sequenceNo
}
```

5. Section 1 — Stats Header

Function: `renderWoSummaryScheduler(json)` | Host div: `#cpo-wo-summary` | View name: `workordersummary`

A 2-row synthetic `Scheduler.getSchedulerInstance()` timeline (NOT a real event calendar — it has no real events, only `cell_template`-rendered HTML per cell). The two rows are "Calculated conclusion" (% coverage + added shortage hours) and "Current position shortage" (hours caused specifically by this Work Order's own current schedule position).

createTimelineView option	Value	Note
render	'bar'	Standard timeline, not tree.
x_unit / x_step / x_size	day / 1 / dates.length	One column per visible day.
y_unit	2 synthetic sections	{key:'fit',...}, {key:'shortage',...} — not real DB rows.
dy	CALC_ROW_HEIGHT = 34px	Identical across all 3 Scheduler instances (1/2/4).
column_width	COLUMN_WIDTH = 92px	DHTMLX auto-stretches columns wider than this unless total content already exceeds container width.
dx	ROW_LABEL_WIDTH = 360px	Row-header width — the shared date-header row sections 1–3 all align under.
scale_height	SCALE_HEIGHT = 36px	
cell_template	true	Every cell's class AND value are custom HTML, computed live.

Data source: nothing from the payload directly — every cell calls `this.evaluateWO(dayIndex)` / `this.currentPositionShortage()[dayIndex]` LIVE at render time (§9), which read `this.db.dayPlanningLines`, `this.db.resources`, `this.db.externalFree`,

`this.db.workOrderSequences`. There is no AL-precomputed "statsRows" array.

> **Confirmed unaffected by the 2026-09-04 redesign**
> The user explicitly reconfirmed mid-session: "the section 1 always run in JS as data already in JS". True before and after §3.1 — renderWorkOrder() (called by BOTH the drag handler and moveWorkOrderToDay, and by the Confirm button's eventual real AL response once that trigger does real work) always calls woSummaryScheduler.setCurrentView(...) to force this section's cell templates to re-read whatever woAnchor is current, purely client-side.

6. Section 2 — Work Order's Own Scheduler

Function: `renderWoScheduler(json)` → `renderWorkOrder()` | Host: `#cpo-wo-scheduler` (wrapped by `#cpo-wo-scheduler-wrap`, §13.5) | View: `workorder`

A real Scheduler timeline with native drag-move enabled. One row per distinct Skill+Job+Task+SequenceNo combination the inspected Work Order itself demands (from `workOrderSequences[]`). Each row's events are positioned via a workday-offset formula (`idxWork(woAnchor, offset)`), not raw calendar "Plan Date" placement — dragging a bar shifts a single client-side `woAnchor` integer and re-renders the whole section instantly (optimistic, pure client-side simulation matching the reference exactly).

createTimelineView option	Value	Note
render	'bar'	
y_unit	one section per sequence	Built from json.workOrderSequences; falls back to a single "No Day Planning lines" placeholder row if empty.
dy / event_dy	32 / 24px	ROW_HEIGHT / ROW_HEIGHT - 8.
dx	360px	Same ROW_LABEL_WIDTH as section 1 (shares its date header).
scale_height	0	Deliberately zero — shares section 1's header row. DHTMLX's own empty header row is force-hidden via CSS.
config.drag_move	true	The only section with native drag.

Events (drag / click) — updated 2026-09-04:

Event	Behavior
onEventChanged (drag)	Computes the day-shift from the drop position, updates woAnchor if valid, resets anchor-dependent caches, re-renders (Sections 1-3), calls markUnconfirmedChange(). **No AL call** — was InvokeExtensibilityMethod('OnRescheduleWorkOrder', ...) on every drag before §3.1; the actual BC round-trip now only happens once, from the Confirm button.
onClick	Opens the (still-stubbed) Capacity Lookup for that event's skill/day — openCapacityLookup(dayIdx, skill). Unaffected by §3.1 (a view action, not a schedule edit — see §3.1's closing note and the user's explicit confirmation to leave it as an immediate call).

Data source: `json.workOrderSequences[]` for row structure; per-cell values pull from `workOrderAssignmentState`, which reads `dayPlanningLines[]` filtered to `line.workOrderNo === inspectedWorkOrderNo` (§11/§13.3 for why this must be the normalized `workOrderNo`, not always the raw table field).

Bug fixed 2026-09-03

This section used to only pick up rows whose raw "Work Order No." table field equalled the inspected WO — but some of a Work Order's own genuine Day Planning Sequences (e.g. added via the Workorder Card's native "New sequence" button) can carry a blank value in that field. AL now scopes by Job No. = WorkOrder."Project No." AND Job Task No. = WorkOrder."Project Task No." instead.

NOT DHTMLX — hand-rolled HTML/CSS

7. Section 3 — Capacity vs Requested Daily Bars

Function: `renderCapacityBars(json)` | Host: `#cpo-capacity-bars`

This section is deliberately not a DHTMLX widget at all — it is plain generated HTML/CSS: for each visible day, a stacked "C" (Capacity: Assigned → Internal free → External free) column and a stacked "R" (Requested: Assigned → Unassigned-per-skill) column, plus Totals/Expand/Exp. to Task/Collapse buttons that control Section 4's tree open state (pure client-side, no AL round-trip). Column width is measured, not the literal COLUMN_WIDTH constant.

Data function	Reads	Produces
capParts(dayIndex)	dayPlanningLines[] (ALL work orders, unfiltered), resources[].length × baseCapacity, externalFree[dayIndex]	{assigned, freeInt, freeExt} — the "C" column. Company-wide on purpose.
dailyCapacityRequestData()	capParts() + dayPlanningLines[] filtered to line.workOrderNo === inspectedWO	One row per visible day: assigned, freeInt, freeExt, request, assignedRequest, unassignedBySkill{}, shortage.

Click-to-relocate drives Section 2 — updated 2026-09-04

Clicking any non-weekend day cell in this section (not a double-click, which opens Capacity Lookup instead) calls `moveWorkOrderToDay(dayIndex)`: sets woAnchor directly to that day, re-renders Sections 1-3, and calls markUnconfirmedChange() — exactly like a Section 2 drag, just reached by a click here instead. As of §3.1, this is pure client-side with **no AL call at all** until the user clicks Confirm; before this session it fired InvokeExtensibilityMethod('OnRescheduleWorkOrder', [dayIndex, ...]) on every click.

Why the "R" (Requested) side IS work-order-filtered but the "C" (Capacity) side is NOT (§11): Section 3 answers "how does this Work Order's own demand sit against the company's real, fully company-wide capacity picture" — the capacity side was already correct before the cross-WO change; only the demand side needed an explicit filter once dayPlanningLines[] started carrying every other Work Order's rows too.

8. Section 4 — Skill → Job/Task → Sequence Tree

Function: `renderCentralTree(json)` | Host: `#cpo-central-tree` (wrapped by `#cpo-central-tree-wrap`, §13.5) | View: `centraltree`

The third real Scheduler instance, using `render:'tree'` (the treetimeline render mode). Three levels: Skill (folder) → Job/Task "detail" row (folder) → Sequence (leaf). Leaf day-cells can render multiple side-by-side "chip" cells when several real Day Planning lines land on the same day/sequence; parent Skill/Task rows show one aggregated heat-map percentage cell instead.

createTimelineView option	Value	Note
render	'tree'	Only section using tree mode.

createTimelineView option	Value	Note
y_unit	buildCentralSections() output	Client-built Skill→Task→Sequence hierarchy from groups[] + dayPlanningLines[].
dy / folder_dy	ROW_HEIGHT = 32px (both)	
section_autoheight	false	Intent: literal row height. In practice DHTMLX still fits total row height to its own init-target element's clientHeight — §13.5's bug.
dx / header width	TREE_LABEL_WIDTH = 360px (3-column: Skill/Job/Task)	Wider than sections 1-3's shared row label — a real 3-column grid header, own independent date row.
columns[0].template	centralTreeLeftColumnHtml(o)	Renders the 3-column Skill|Job|Task left panel per row.

Per-row/cell templates:

- `centraltree_cell_value` — Skill rows call `skillDaySummary()`; Job/Task "detail" rows call `taskDaySummary()` (both render a gradient-filled % heat cell); Sequence leaf rows call `sequenceDayCellHtml()` (renders the actual chip(s)). As of 2026-09-04 (§13.8), all three read from a precomputed `treeSummaryIndex()` instead of rescanning dayPlanningLines[] on every call.
- Chips are clickable/hoverable — `attachTreeChipTooltip()` binds one shared mouseover/click handler on the whole host div, reusing the same tooltip element Section 2 uses. Click raises `OnSequenceChipClick` (still an immediate AL call, see §3.1's closing note).

Bug fixed 2026-09-03 (scope)

groups[] (and therefore this whole tree) is built AL-side exclusively from every OTHER Work Order's demand in the visible window — the inspected Work Order's own data must never appear here. The exclusion criterion had to be corrected from the raw "Work Order No." field to a Job No./Job Task No. match against the inspected Work Order.

Bug fixed 2026-09-03 → follow-on fixed 2026-09-04 (V-scrollbar)

The ORIGINAL bug (§13.5): DHTMLX was auto-shrinking every row's rendered height to fit its own init-target element's reported height, so a tree with far more rows than fit never actually overflowed and no scrollbar ever appeared — fixed by splitting the host into an unconstrained inner div (DHTMLX's real render target) wrapped by an outer div carrying the MIN/MAX-clamped height + overflow-y:auto. **The FOLLOW-ON bug found 2026-09-04**: that height was only ever applied ONCE, at the initial render — collapsing/expanding rows afterward left the wrapper sized for whatever row count was visible last, so the scrollbar thumb kept spanning the fully-expanded range and a blank gap appeared below the actual (now shorter) row list. See §13.7 for the fix and live confirmation.

Performance fixed 2026-09-04

skillDaySummary/taskDaySummary/sequenceDayLines each used to rescan the ENTIRE dayPlanningLines[] array on every single rendered cell, making Expand/Collapse-all multi-second on real data even though nothing round-trips to BC. See §13.8 for the treeSummaryIndex() fix and live-measured before/after numbers, and §13.11 for why both skill and Job/Task folders now also start CLOSED on load rather than expanding that cost automatically.

9. The Shortage / Coverage Engine (Max-Flow)

Section 1's two rows and Section 3's implicit "shortage" figure are computed by a near-verbatim port of the reference prototype's own client-side algorithm — an Edmonds-Karp-style augmenting-path max-flow model (`maxFlowDay`), not a simple "sum capacity vs sum demand" formula.

Function	Purpose
maxFlowDay(dayIndex, demandBySkill)	Given one day's per-skill demand, computes the maximum achievable coverage across the (skill-scoped, capped) resource pool via flow conservation — achievable coverage is capped by demand, not supply.
evaluateWO(dayIndex)	"What if this Work Order's current schedule position is included?" — returns {coverage, addedShortage}, the source of Section 1's "Calculated conclusion" row.
currentPositionShortage() / currentPositionSkillShortage()	Before/after delta: maxFlowDay WITHOUT this WO's demand vs WITH it, at its current woAnchor position — the source of Section 1's "Current position shortage" row and Section 2's per-row shortage badges.

> **Why the algorithm was reversed back in (2026-09-02 decision)**
>
> An earlier, simplified "sum team capacity minus other-WO-assigned-hours" formula had a real bug — a large company-wide resource pool produced coverage numbers over 5,700%. The max-flow model doesn't have this failure mode because achievable throughput is fundamentally bounded by demand, never by an oversized supply pool.

Performance caveat: max-flow is expensive (roughly O(resources²) per day) and is called from BOTH Section 1's per-cell templates AND the shortage functions' own per-day loops, with no memoization in the reference. This port adds per-render-pass caching (`this._evalWOCache`, `this._currentPositionShortageArr`, cleared on every `applyPlanningData()` / `resetAnchorDependentCaches()`) and AL caps the resource pool at 15 per skill (`CPO_MaxResourcesPerSkill`) — both are deliberate, documented performance trade-offs, not data-completeness bugs. This caching is unaffected by the 2026-09-04 redesign: resetAnchorDependentCaches() is still called on every local reschedule move, before or after §3.1 — only the AL call at the end of that path changed.

10. AL-Side Implementation Details

codeunit 50604 "DHX Data Handler" — CPO_ region, key procedures

Procedure	Purpose
CPO_BuildPlanningDataJson_Paged(WorkOrderNo, NumberOfDays, MaxOtherLines, out RemainingGroupKeys): Text	The one entry point — assembles the entire payload in the 3-pass + pagination shape described in §12.
CPO_BuildDayPlanningLineObj(DayPlanning, InspectedWorkOrderNo, InspectedJobNo, InspectedJobTaskNo): JsonObject	One dayPlanningLines[] entry. Normalizes workOrderNo to the inspected WO's own No. whenever the line's Job No./Job Task No. match it.
CPO_BuildSkillsArray / CPO_BuildResourcesArray / CPO_BuildExternalFreeArray / CPO_BuildGroupsArray	Build skills[] / resources[] / externalFree[] / groups[] respectively — §4 for each one's exact scope.
CPO_BuildWorkOrderSequencesArray	Builds workOrderSequences[] from the Pass-1 dedup lists.
CPO_ComputeWorkdayOffset / CPO_IndexOfText / CPO_BuildDateRangeArray / CPO_FormatTimeHHMM	Small shared helpers (offset math, list lookup, date range, time formatting).
CPO_MaxResourcesPerSkill(): Integer	Returns 15 — the max-flow performance cap (§9).

page 50722 "Capacity Planning Overview"

- No SourceTable — a bodiless Card page holding exactly one usercontrol.
- Caption = ''; — blank on purpose (the JS renders its own title bar).
- SetWorkOrderNo(pWorkOrderNo) — filter-setter-before-Run(), called by the launching page before opening.
- DaysToShow global var, default 30 (DefaultDaysToShow()) — page-level state; the reschedule flow's pending "unconfirmed change" state (2026-09-04) lives entirely client-side, NOT mirrored into any AL variable.
- RefreshData() — single shared rebuild-and-push routine called by ControlReady and OnDaysToShowChanged.
- **OnRescheduleWorkOrder(DayShift, PayloadJsonTxt)** — unchanged signature; 2026-09-04 removed its Message('Reschedule stub: shift=%1', DayShift) body (§13.10), now a true silent stub matching OnRequestCapacityLookup/OnSequenceChipClick. Only ever invoked from the Confirm button now (§3.1), not per drag/click.

Pag50662.WorkorderCard.al — launching action

```
action(CapacityPlanningOverviewAct)
{
    trigger OnAction()
    var CPO: Page "Capacity Planning Overview";
    begin
        CPO.SetWorkOrderNo(Rec."Work Order No.");
        CPO.Run();          // plain top-level navigation - see 13.6
    end;
}
```

11. Cross-Work-Order Scoping (2026-09-03 Correction)

Explicit product decision, stated by the user in these exact terms: "section 3 for DWO0008 but section 4 for all day planning data in days time frame except DWO0008", further clarified as scoped "as long as in same time frame of days" and explicitly including "other WO in other job" (i.e. truly company-wide, not scoped to the inspected WO's own parent Job).

Section	Scope	Rationale
Section 1 (stats)	Inspected WO only	"This WO's own position." Literally IS the inspected WO's own schedule.
Section 2 (own scheduler)	Inspected WO only	"This WO's own demand."
Section 3 — "R" (Requested)	Inspected WO only	A resource committed elsewhere that day is genuinely unavailable — already correct before this change.
Section 3 — "C" (Capacity)	Company-wide (unfiltered)	"Everything else going on in this window, for contention/context."
Section 4 (tree)	Every OTHER Work Order, ANY Job, same date window	Intentionally the opposite of the earlier, narrower behavior ("this WO's own tree") — explicit and permanent.

Still true and unchanged by the 2026-09-04 confirm-gated redesign: nothing about §3.1 alters WHICH data each section shows, only WHEN (if ever) a reschedule action tells BC about a change — Section 4's scope in particular is completely untouched, since it was already never re-rendered by any reschedule path.

12. Performance — Page Background Task Pagination

Added 2026-09-03, on explicit user request. As Section 4's cross-Work-Order scope (§11) made the add-in query every other Work Order's Day Planning demand company-wide for the visible window, real companies with a large Day Planning volume for that period made JSON generation (AL) and DHTMLX ingest (JS parse + tree/model build + render) slow enough to notice on first paint. The fix ports this repository's existing Page Background Task pattern — already used by request_assignment (codeunit 50721), projectschedule (codeunit 50720), and ganttdemo2 (codeunit 50713) — into CPO, following the exact same shape.

12.1 The technique (Business Central Page Background Task)

API	Role
CurrPage.EnqueueBackgroundTask(TaskId, Codeunit, Parameters, TimeoutMs, ErrorLevel)	Starts a child session running the given codeunit's OnRun trigger asynchronously, off the interactive request path. Returns immediately.
Page.GetBackgroundParameters(): Dictionary of [Text, Text]	Read inside the child codeunit's OnRun to retrieve the parameters passed at enqueue time.
Page.SetBackgroundTaskResult(Dictionary of [Text, Text])	Called at the end of the child codeunit's OnRun to hand results back to the parent page.
trigger OnPageBackgroundTaskCompleted(TaskId, Results)	Fires on the parent page when the child session finishes. Cannot safely call the control add-in from here — must stash the result into a plain AL variable instead.
trigger OnPageBackgroundTaskError(TaskId, ErrorCode, ErrorText, ErrorCallStack, var IsHandled)	Fires on failure/timeout. Surfaced via a Notification, not a blocking error.

Why results can't be pushed to the control add-in directly from `OnPageBackgroundTaskCompleted`: confirmed live across four independent add-ins in this repository — BC Server rejects a control add-in callback issued directly from that trigger. The workaround: stash the result into a plain page variable, then let a JS-initiated poll (a normal synchronous trigger call, safe to answer with a control add-in callback) pick it up and deliver it.

12.2 CPO's architecture (mirrors request_assignment's board exactly)

```
page 50722 RefreshData()
  CPO_BuildPlanningDataJson_Paged(WorkOrderNo, DaysToShow, MaxOtherLines=50, out RemainingGro
upKeys)
  CurrPage.DhxCpo.SetPlanningData(json)                 // first paint: fast
  EnqueueOtherWorkOrderDataBackgroundTask(RemainingGroupKeys)
     CurrPage.EnqueueBackgroundTask(.., Codeunit::"CPO BG Other WO Data", params, 30000, ..)
     CurrPage.DhxCpo.NotifyOtherWorkOrderDataTaskPending() // JS starts polling + shows #cpo-
bg-loading (3.2)

(async, off the interactive request path)
codeunit "CPO BG Other WO Data".OnRun()
    CPO_BuildOtherWorkOrderLinesJson_ForKeys(WorkOrderNo, StartDate, EndDate, RemainingGroupK
eys)
    Page.SetBackgroundTaskResult({ otherWorkOrderDataJson: json })

trigger OnPageBackgroundTaskCompleted(TaskId, Results)
    stash Results into PendingOtherWorkOrderDataJson (plain var - no control add-in call here
)

trigger OnPollOtherWorkOrderDataResult()              // JS-initiated, polled every 500ms
    CurrPage.DhxCpo.AppendOtherWorkOrderData(PendingOtherWorkOrderDataJson)
    CurrPage.DhxCpo.StopOtherWorkOrderDataPolling()   // also hides #cpo-bg-loading (3.2)
```

12.3 The atomic pagination unit — whole Skill+Job No.+Job Task No. groups, never split

Exactly like request_assignment's sequenceKey pagination, CPO never splits a rendering unit across the synchronous first page and the background remainder. The unit here is a whole group — Skill+Job No.+Job Task No., the same combination Section 4's tree groups by.

Pass	Cost	Scope	Produces
CPO_ScanOtherWorkOrderGroups (Pass 3a)	Cheap — key fields only, no per-line JSON	ALWAYS full — every group, never paginated	Complete skills[]/resources[]/externalFree[]/groups[] + the group order/line-count needed to decide the page cutoff
CPO_BuildOtherWorkOrderLinesForGroups (Pass 3b)	Expensive — lookups, time-to-decimal, full per-line JSON	Paginated — only whichever whole groups are in WantedGroupKeys	dayPlanningLines[] entries — the expensive part, and the only part actually deferred

Practical effect: Section 4's tree skeleton renders complete on first paint, even over a huge dataset. Only the Sequence-level leaf chips for groups past the first 50 backfill progressively once the background task delivers them, typically within the same second or two on this demo dataset (confirmed live: 1,388 distinct groups across 8 skills, 5,042 total Day Planning lines merged successfully with zero console errors).

12.4 Why Section 1/2/3 are treated differently

Section	Paginated?	Reason
Section 2 (inspected WO's own scheduler)	No — Pass 1/2 unchanged	Bounded to one Work Order's own lines — never the actual bottleneck.

Section	Paginated?	Reason
Section 1 (stats) / Section 3 (capacity bars)	Indirectly — provisional until backfill completes	Computed from the full company-wide dayPlanningLines[]. Correct for whatever data has arrived so far, self-corrects once the remainder appends.
Section 4 (tree)	Yes — the actual target	Skeleton always complete; chip-level detail backfills per §12.3.

12.5 The JS-side append — why it's more than a tree rebuild

CPO's shortage/coverage engine (§9) and Section 3's capacity bars are computed from the full dayPlanningLines[] set, so `appendOtherWorkOrderData(rawLines)` must: (1) push the new lines into `this.db.dayPlanningLines`; (2) invalidate every derived cache (including, as of 2026-09-04, `this._treeSummaryIndex` — §13.8); (3) re-render Sections 1, 3, and 4 and re-bind the shared scrollbar. Section 2 is not re-rendered — its own source data was never paginated.

12.6 New/changed objects (2026-09-03)

Object	Change
codeunit 50722 "CPO BG Other WO Data"	New. Background task target — modeled directly on codeunit 50721.
codeunit 50604: CPO_BuildPlanningDataJson_Paged	New. Paged variant of CPO_BuildPlanningDataJson.
codeunit 50604: CPO_ScanOtherWorkOrderGroups	New. Pass 3a — the cheap, always-full pre-scan.
codeunit 50604: CPO_BuildOtherWorkOrderLinesForGroups	New. Pass 3b.
codeunit 50604: CPO_BuildOtherWorkOrderLinesJson_ForKeys	New. Background-task companion.
page 50722	RefreshData() now calls the paged builder + EnqueueOtherWorkOrderDataBackgroundTask; new background-task triggers.
Controladdin	New procedures/events for the pagination poll.
wrapper.js	New poll timer + window.AppendOtherWorkOrderData.
capacityPlanningOverview.js	New appendOtherWorkOrderData(rawLines) method.

13. Known Pitfalls, Bugs Found & Fixes Applied (session log)

Renumbered 2026-09-04 — this section's entries were previously mislabelled 12.1–12.6 (reusing §12's numbers, a copy-paste artifact of the original document); they are correctly 13.1–13.6 below, with the current session's findings added as 13.7–13.11.

13.1 Unterminated CSS comment silently disabled the top bar

style.css had a comment inside #cpo-root {} that opened (/*) but never closed before that block's own }, so the first real */ the parser then found was ~30 lines later, silently swallowing .cpo-top-bar's and .cpo-top-title's entire rule bodies as dead comment text. Symptom: the title and "Days to show" input stacked on two plain, unstyled rows instead of one styled row. Fix: add the missing */ immediately after the intended one-paragraph comment.

Lesson

An unterminated CSS comment doesn't error — it silently consumes everything until the next literal */ anywhere later in the file. Always grep for balanced /* / */ pairs when a rule that "should" apply visibly doesn't.

13.2 RunModal() vs Run() — native BC chrome

Opening the page via RunModal() always renders BC's own dialog chrome (bold title bar + command bar + Close button) regardless of the page's own Caption or field content. Switching to Run() (plain top-level navigation, the same pattern page 50710 "DHX Request Assignment Board" already used) replaces it with just a back arrow, matching the reference prototype's own minimal chrome.

13.3 "Work Order No." field vs Job No./Job Task No. scoping

See §6/§8/§11. The native Workorder Card's own "Day Planning Sequence" part filters Day Planning by Job No./Job Task No., never by the "Work Order No." field. Some genuine Day Planning Sequences can have that field blank. Any AL query that scopes "this Work Order's own data" by the "Work Order No." field alone will silently miss such rows — and symmetrically, any exclusion keyed the same way will wrongly include them as if they belonged to someone else.

13.4 Flexbox default shrink crushed the shared scrollbar

#cpo-root is a fixed-height display:flex; flex-direction:column container. #cpo-shared-scroll (declared height:16px) had no flex-shrink override, so whenever the other sections' combined natural height exceeded #cpo-root's own bounded height, the flex column's default flex-shrink:1 proportionally squashed every child — measured live: rendered height collapsed to 1px. Fix: flex: 0 0 16px; (no shrink, no grow, fixed 16px basis).

13.5 DHTMLX auto-fits row height to its container — scrollbars never appear (original fix)

Even with dy/folder_dy fixed and section_autoheight:false, DHTMLX Scheduler reads its own init-target element's clientHeight at render time and fits all row content to exactly that height (shrinking rows) rather than rendering at the literal configured height and overflowing. Fix (Sections 2 and 4): split the single host div into two nested divs — DHTMLX initializes into an inner div left height-unconstrained, while a new outer wrapper div carries the MIN/MAX-clamped height and overflow-y:auto that actually does the visual clipping/scrolling. See §13.7 for the 2026-09-04 follow-on to this fix.

13.6 Concurrent publish/build collisions

Running multiple al_publish operations from parallel sessions/subagents against the same sandbox environment can produce a spurious CompilationFailed even when al_getdiagnostics shows zero real errors — a file-lock/build collision, not a code defect. Resolved by waiting for the other publish to finish and retrying.

13.7 Section 4 scrollbar drifted from the actual row composition after collapse/expand

New, 2026-09-04. §13.5's fix solved the INITIAL scrollbar bug, but applyCentralTreeHeight was only ever called once, at the initial renderCentralTree pass — collapsing/expanding rows afterward (via setTreeOpenState's Expand/Collapse-all buttons, or a single row's own native fold arrow) never re-ran it, so the wrapper's clip height stayed sized for whatever row count was visible at the LAST full render. Symptom (reported by the user with a screenshot): collapse all rows, and the scrollbar thumb still spans the fully-expanded range, leaving a blank scrollable gap below the now-much-shorter row list.

Fix

bindCentralTreeHeightSync listens for DHTMLX's onOptionsLoad event (fired both by setTreeOpenState and by a single row's own native toggle) and re-applies applyCentralTreeHeight against the LIVE tree state every time. setTreeOpenState additionally pre-applies the height BEFORE firing onOptionsLoad, so DHTMLX's own native redraw already reads the corrected height on its one pass — no forced second redraw needed (an earlier attempt added an explicit setCurrentView() call here on the theory that DHTMLX's native handler might read a stale height otherwise; §13.8 explains why that call was removed again).

Confirmed live

Collapse-all shrank the wrapper's scrollHeight from 90,476px to 300px (the MIN_TREE_HEIGHT clamp); a single row's own toggle correctly produced an intermediate 24,012px; Expand-all scrolled cleanly to the true last row with no blank gap in all three cases.

13.8 O(rows × days × dayPlanningLines) per-cell rescans made Expand/Collapse-all multi-second

New, 2026-09-04 — raised by the user as "as Data already in JS, why take long time during expands and collapse?". skillDaySummary/taskDaySummary/sequenceDayLines each did a full .forEach/.filter scan over the ENTIRE dayPlanningLines array (this WO's lines plus every other WO's — §11) — and DHTMLX calls each of these once per RENDERED CELL (every visible row × every visible day) during its native tree redraw. Real cost: O(rows × days × dayPlanningLines.length) — genuine, user-visible latency purely on the client, unrelated to any AL round-trip.

A wrong first diagnosis, corrected by profiling

The §13.7 fix's own earlier draft added an explicit setCurrentView() call on the theory that it was needed for correctness. When THIS bug was investigated, profiling with that call instrumented (not yet removed) showed it consuming ~5.4s of a ~5.4s total cost — but removing it entirely and re-profiling showed the SAME ~6.3s cost still fell inside DHTMLX's own native redraw call, proving the extra call was never the real bottleneck. Lesson (explicitly worth keeping): attribute a regression to a specific recent change only after profiling the counterfactual, not by inspection or recency alone.

Fix

treeSummaryIndex() groups dayPlanningLines ONCE per data load (O(dayPlanningLines.length) total, invalidated on the same applyPlanningData/appendOtherWorkOrderData calls that already reset the other derived caches, §9) into skill+day / skill+job+task+day / skill+job+task+sequence+day lookup maps. The three summary functions became O(1) map lookups.

Confirmed live

On DWO0006's real data, Expand-all dropped from ~5.4-6.3s to ~1.7s (≈3.5×), with identical output (same hour totals, no squished/misaligned rows, correct scrollbar behavior per §13.7).

13.9 InvokeExtensibilityMethod is fire-and-forget from JS's own perspective

New finding, 2026-09-04 — discovered while investigating §13.7/§13.8 and directly informing the §3.1 redesign and §3.2's indicator design. Confirmed live by wrapping the call with performance.now() timing: the JS call itself returns in well under 1ms regardless of what the AL trigger it targets actually does — even a genuinely slow AL round-trip (a "Days to show" reload rebuilding company-wide data took ~82s wall-clock in one measurement) does not block the calling JS thread. The real response, if any, always arrives later through a SEPARATE AL-to-JS call (SetPlanningData, AppendOtherWorkOrderData, LoadCapacityLookup), never as a return value from the invoke itself.

Practical effect on this add-in's own design

showLoading() called immediately before InvokeExtensibilityMethod always gets a real paint for free, since the call yields control back to the browser well before AL finishes — no requestAnimationFrame deferral trick needed there (openCapacityLookup, the Section 4 chip click, confirmChanges all rely on this). That trick (runBusy, §3.2) is needed ONLY around genuinely synchronous CPU-bound client work with no natural browser yield point — DHTMLX's own tree redraw (§13.8), specifically.

13.10 Reschedule stub feedback: blocking dialog → JS notice → removed entirely for a Confirm button

New, 2026-09-04. OnRescheduleWorkOrder's stub originally proved its wiring fired via a native Message('Reschedule stub: shift=%1', DayShift) dialog. That was reasonable when reschedule was a rare, deliberate action — but at the time every single drag/click-to-relocate fired it, interrupting the user with a mandatory OK click on every intermediate step (screenshot evidence: a dialog stacked directly over the schedule mid-drag).

Iteration 1 — non-blocking JS notice

Removed the AL Message() call, replaced with a small non-blocking JS-rendered notice (a bordered text box in Section 3's fixed left column, auto-hiding after 4s) showing the same "Reschedule stub: shift=N" text, computed entirely client-side (the shift value was already known in JS before ever invoking AL).

Iteration 2 (final) — removed entirely

Once reschedule became fully local-simulation + Confirm-gated (§3.1), the per-action notice's whole reason for existing disappeared too — there is no longer a per-action AL call to "prove fired". Removed; the Confirm button's own enabled/pending visual state (.cpo-confirm-btn-pending) is what now signals "you have an unconfirmed move" instead.

13.11 Expand-all still felt slow after §13.8 — three dead ends before the real fix

New, 2026-09-04 — even after treeSummaryIndex (§13.8) cut Expand-all to ~1.7-2.1s, the user kept pushing on "it must be faster, data is already in DHTMLX" (fair — 1.7s still reads as sluggish for a purely local operation). Three approaches were tried and profiled live before landing on the one that actually worked.

Dead end 1 — DHTMLX Scheduler's own smart_rendering option

Scheduler's minified bundle ships a viewport-based lazy-render path (_timeline_smart_render.getViewPort). Enabling smart_rendering: true on the centraltree view was a natural next try — measured live, it made Expand-all SLOWER (~8.1s, not faster). Reverted. Likely not effective for render:'tree' mode's folder hierarchy the way it is for a flat timeline.

Dead end 2 — swap Section 4 to the DHTMLX Gantt library

The Gantt add-in's (ganttdemo2) own collapse/expand (gantt.eachTask(t=>t.$open=open); gantt.render()) is fast because DHTMLX Gantt virtualizes rows natively. Investigated as a full swap; found two real obstacles first: Gantt's LEFT grid/tree panel does not scroll horizontally on its own (only its chart/timeline pane does), and that chart pane draws task BARS positioned by date range, not arbitrary per-cell HTML the way Scheduler's cell_template does — Section 4's heat-cells/chips would need to become day-wide bars to fit that model. Surfaced to the user as a scoped decision before any code was written; the user chose to stay on Scheduler rather than take on that rewrite.

Dead end 3 — "copy Task Scheduler's mechanism"

The user pointed at Task Scheduler (projectschedule) as an example of fast expand/collapse still on DHTMLX Scheduler. Reading its actual ToggleCollapseExpandAllSections in projectschedule/wrapper.js showed it is BYTE-FOR-BYTE the same technique CPO's own setTreeOpenState already uses (same y_unit_original mutation, same _getArrayToDisplay, same onOptionsLoad call) — CPO's own code comment already says it was ported from that exact function. There was no different mechanism to copy. The real gap is scale: Task Scheduler's tree is Job→Job Task (scoped, few rows); CPO's Section 4 fully expanded to Sequence level, company-wide (§11), is 2,831 rows — DHTMLX's native redraw cost after onOptionsLoad scales with row count regardless of which code toggles the open flags.

Fix — start collapsed, pay the cost only when asked

buildCentralSections() now sets open: false for BOTH the skill and the Job/Task "detail" level on every render pass, regardless of AL's own groups[].expanded flag (which drove the skill level's initial state before). Expand-all's ~1.7-2.1s is real, unavoidable-at-that-scale DOM-construction cost for 2,831 rows — the fix is not making that number smaller, it is making sure the user only ever pays it when they actually click Expand, not on every page open.

Confirmed live

Fresh page load: all 8 skill rows render collapsed instantly (wrapper clientHeight/scrollHeight both 300px, the MIN_TREE_HEIGHT clamp - no scrollbar needed at all). Expand-all afterward: scrollHeight grows to 90,636px in ~2.16s (the §13.8-era cost, now opt-in only), and scrolling to the bottom lands exactly at the true last row with no blank gap - confirming §13.7's scrollbar-sync fix still holds with the new default state.

14. Appendix — File Reference

Path (relative to repo root)	Contents
src\dhx\capacity_planning_overview\DHXCapacityPlanningOverviewAddin.ControlAddin.al	Controladdin declaration
src\dhx\capacity_planning_overview\page_50722_CapacityPlanningOverview.al	Host page 50722
src\dhx\capacity_planning_overview\startupScript.js	BOOT() call
src\dhx\capacity_planning_overview\wrapper.js	BOOT/binding shell + loading-overlay safety timer (§3.2)
src\dhx\capacity_planning_overview\capacityPlanningOverview.js	The single JS component (~1,850 lines) — all 4 sections + shortage engine + Confirm-gated reschedule state machine (§3.1) + processing indicators (§3.2) + treeSummaryIndex (§13.8)
src\dhx\capacity_planning_overview\style.css	All layout/heat-map/chip/scrollbar/spinner/Confirm-button CSS
src\codeunit\codeunit_50604_DHXDataHandler.al	Shared data-handler codeunit — CPO_ region builds the payload
src\dhx\capacity_planning_overview\codeunit_50722_CPOBGOtherWorkOrderData.al	Page Background Task target — §12
src\page\Pag50662.WorkorderCard.al	Launching action CapacityPlanningOverviewAct
src\dhx\dhtmlxscheduler.js / .css	Shared vendor Scheduler library (root-level, reused by every DHX add-in)
src\dhx\suite.js / .css	Shared vendor Suite library (declared but not currently used by this add-in's own modal, which is stubbed)
C:\Users\Ahmad\OneDrive\x\PROJECT\KLAAS\CPO_v131\DHTMLXtempv112-app.js	Reference prototype this add-in is ported from (external to this repo)

End of document.